Late Breaking Results: A Fast Nearest Neighbor Search Acceleration for 3D Point Cloud

Jinao Li†, Teng Wang*‡, Qianyu Cheng†, Zhendong Zheng†, Lei Gong†, Chao Wang†‡, Xi Li†‡, Xuehai Zhou†‡
† *School of Computer Science and Technology, University of Science and Technology of China*, Hefei, China
‡ *Suzhou Institute for Advanced Research, University of Science and Technology of China*, Suzhou, China
{lijinao,qycheng,zzd1411}@mail.ustc.edu.cn
{wangt635,leigong0203,cswang, llxx, xhzhou}@ustc.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133158

***Abstract*—This paper presents FastNN, a novel accelerator architecture for efficient K-Nearest Neighbors (KNN) search in point clouds. FastNN leverages a locality-sensitive E2LSH partitioning method and a pre-comparator module to significantly reduce the candidate search space and minimize the number of Euclidean distance calculations. Compared to octree-based partitioning methods, our approach reduces candidate points by 58.57% to 86.17% and achieves a 10.04× acceleration in processing throughput relative to the BitNN comparator subsystem. The proposed design effectively enhances search throughput, resource utilization, and precision, highlighting its potential for accelerating KNN search on FPGA platforms.**

***Index Terms*—3D Point Cloud, Hardware Acceleration, Nearest Neighbor Search**

I. Introduction

3D Point cloud data is commonly acquired using sensors such as LiDAR(Light Detection and Ranging), depth cameras, and stereo vision systems. It has found widespread applications in various fields, including computer vision, robot navigation, 3D modeling, and autonomous driving [1].

The K-Nearest Neighbor(KNN) search plays a pivotal role in point cloud processing, accounting for over 70% of the computation time [2]. However, due to the memory-intensive nature of the KNN algorithm and the bandwidth limitations of the application platform, it is currently challenging to meet throughput and latency requirements.

There are two main methods to optimize KNN's latency. The first method uses partitioning algorithms to divide the point cloud into regions [3], allowing each query point to find neighboring references within a specific area. This reduces the search space but requires a trade-off between precision and efficiency. Current partitioning methods often result in long index construction times or leave redundant search spaces that do not sufficiently reduce reference points. The second method focuses on reducing Euclidean distance computation. Studies show that this computation accounts for over 80% of processing time in KNN searches [4], leading to high computational overhead and noticeable delays. Instead of exact calculations, approximating distance and comparing it to the K-th candidate's distance from the previous cycle is sufficient; however, the current method does not effectively filter and eliminate candidate points.

In this work, we present FastNN, an accelerator for KNN search in point clouds that utilizes the locality-sensitive E2LSH partitioning method and pre-comparator, improving the performance of the KNN algorithm in two aspects. Specifically, by using the E2LSH algorithm, we reduce the number of candidate points, achieving an average reduction of 76.71% in candidate points compared to the octree method, thereby reducing off-chip memory accesses. Additionally, by utilizing the pre-comparator, we reduce the number of Euclidean distance calculations, resulting in a 10.04× performance improvement over BitNN [4].

* corresponding author.

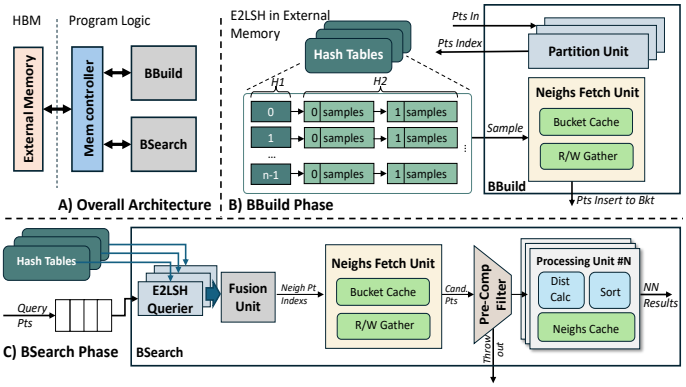

Fig. 1. Architecture overview of FastNN.

II. FastNN Architecture

The overall framework overview is shown in Fig. 1. This framework consists of two main components: the BBuild phase for index construction and the BSearch phase for querying K-nearest neighbors.

A. E2LSH in BBuild

In the BBuild phase, the E2LSH partition method is used to divide the point cloud, as shown in Fig. 2. Points that are relatively close to the original spatial position are assigned to the same bucket, thereby limiting the search space to only the points within the bucket. This eliminates the need to compute and compare the Euclidean distances for every points. A two-level hash table is utilized to store the reference point set, with its hash function defined as shown in (1).

$$h_{ab}(v) = \left\lfloor \frac{a \cdot v + b}{w} \right\rfloor \tag{1}$$

Here, a is a d-dimensional vector, with each dimension independently drawn from a normal distribution, and b is a random number uniformly sampled from $[0, w]$, where w is a user-defined parameter representing the bucket width. The operation $a \cdot v$ maps the feature vector v to the real number set $\mathbb{R}$. By dividing the real axis into intervals of width w and assigning a label to each interval, the hash value is determined by the label of the interval where $a \cdot v$ falls. The two indices of the two-level hash table, H_1 and H_2, are computed using (2) and (3), respectively.

$$H_1(h) = [(r \cdot h) \ mod \ C] \ mod \ tableSize \tag{2}$$

$$H_2(h) = (r' \cdot h) \ mod \ C \tag{3}$$

where r and r' are random integers, and C is a large prime number, which can be set to $2^{32} - 5$ on a 32-bit machine. To improve accuracy,

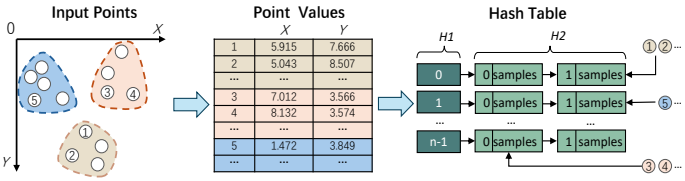

Fig. 2. Example of the E2LSH partitioning process

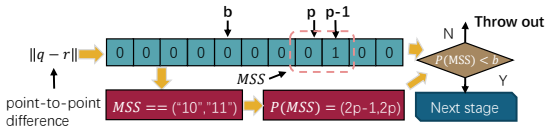

Fig. 3. The architecture of pre-comparator unit

we maintain L hash tables, each constructed using independent hash functions, and transfer these hash tables onto the chip via L on-chip memory channels.

B. Pre-Comparison In BSearch

For the BSearch implementation, the query points are first loaded from off-chip memory to on-chip buffers and then mapped to corresponding hash buckets through identical hash functions. Since points within the same bucket exhibit spatial proximity in the original metric space and the parameter K (typically 5-10) remains small, we compute the intersection of candidate neighbors from L hash tables' corresponding buckets. To minimize off-chip random memory access, we implement a read-gather cache that coalesces queries accessing identical buckets, thereby enhancing data reuse efficiency.

We further devise an early-stage comparator to reduce full Euclidean distance computations, as shown in Fig. 3. We first obtain the most significant bit(MSB) p of the difference in each dimension between the reference point and the query point, as well as the next most significant bit, $p-1$. If the MSS equals "10", then the most significant bit of the estimated distance is determined as shown in (4); if the MSS equals "11", it is determined as shown in (5).

$$P(\texttt{MSS}) = 2p - 1 \tag{4}$$

$$P(\texttt{MSS}) = 2p \tag{5}$$

In each cycle, we cache the most significant bit b of the distance between the current K-th candidate neighbor and the query point. Next, we compare MSS with the b from the previous cycle; if $P(\texttt{MSS}) < b$, we proceed to the next phase to perform a full Euclidean distance calculation, otherwise, we discard it directly. This design leverages the observation that fractional components exert negligible influence on distance comparisons compared to their integer counterparts, enabling a coarse-grained approximation. By projecting the estimated value to position $2p-1$, we apply equivalent decision logic to both integer and fractional components. Upon completion of all processing cycles, the finalized candidate neighbors are written back to off-chip memory as the definitive output. Finally, we discard points with a significant distance discrepancy from the query point, achieving an accuracy of 93%.

III. Experiments & Results

For accelerator implementation, we employed the Xilinx Vitis HLS 2023.1 framework deployed on Xilinx Alveo U280 platforms. A comprehensive evaluation was conducted to quantify performance across critical metrics, including search throughput, resource utilization efficiency, and search precision. To benchmark computational latency, we adopted the KITTI fixed-point dataset as our baseline, where each processed frame contains approximately 50k non-ground points. From this dataset, 500 representative points were selected as reference points with K=5 configured as the neighborhood size.

TABLE I
Comparison of Candidate Point Accesses

Methods	L(Point number=51361)				
ParallelNN [5]	17009	17009	17009	17009	17009
Ours	3531	4006	2353	2873	7047
Reduction	79.24%	76.45%	86.17%	83.11%	58.57%

TABLE II
Comparison with Other Pre-Comparators

	Simple-KNN	BitNN [4]	Ours
LUTs	4323	17697	3544
FFs	6226	8892	3695
BRAMs	30	21	16
DSPs	3	89	3
Search Latency	76107	35131	3499

We compared the number of points traversed in the software when using E2LSH partitioning versus when it is not used, as quantitatively validated in Table I. Compared to the partitioning method used by ParallelNN [5], our approach reduces the number of candidate points by 58.57% to 86.17%. Furthermore, when benchmarked against BitNN's comparator subsystem under equivalent dataset and hardware configurations, our design achieves a 10.04× acceleration in processing throughputs, as shown in Table II. Due to the limited size of the current dataset, the parallelism of the BitNN pre-comparator could not be fully exploited, resulting in suboptimal performance.

IV. Conclusion & Future Work

In this paper, we propose a novel accelerator architecture for KNN and demonstrate its significant advantages in terms of memory footprint and acceleration performance. Currently, our pre-comparator employs a bit-by-bit search to identify the most significant bit (MSB), an approach that is inefficient. In future work, we plan to adopt a non-uniform block partitioning method—partitioning the higher bits into coarse-grained blocks and the lower bits into fine-grained blocks. This strategy will help us locate the MSB more quickly and enhance parallel processing capability.

Acknowledgment

This work was supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDB0660101, XDB0660000, and XDB0660100, in part by the National Natural Science Foundation of China under Grant and 62172380, in part by Jiangsu Provincial Natural Science Foundation under Grant BK20241818, in part by Youth Innovation Promotion Association CAS under Grant Y2021121, in part by USTC Research Funds of the Double First-Class Initiative under Grant YD2150002011.

References

[1] A. Asvadi and et al., "3d lidar-based static and moving obstacle detection in driving environments," *Robot. Auton. Syst.*, vol. 83, no. C, p. 299–311, Sep. 2016.
[2] Y. Feng and et al., "Mesorasi: Architecture support for point cloud analytics via delayed-aggregation," in *MICRO'20*, pp. 1037–1050.
[3] R. Pinkham and et al., "Quicknn: Memory and performance optimization of k-d tree based nearest neighbor search for 3d point clouds," in *HPCA'20*, pp. 180–192.
[4] M. Han and et al., "Bitnn: A bit-serial accelerator for k-nearest neighbor search in point clouds," in *ISCA'24*, pp. 1278–1292.
[5] F. Chen and et al., "Parallelnn: A parallel octree-based nearest neighbor search accelerator for 3d point clouds," in *HPCA'23*, pp. 403–414.